PROJECT REPORT

Supervisor

Mr Mehtab Mushtaq

Course

Operating system

Project

Deadlock Detection Simulator

Submitted To

Mr Mehtab Mushtaq

Submitted by

Kaleem Faryad (051)

Muhammad Mudasir Hafeez (050)

Alyan Jahangir (033)

Session 2023-2027

Date of submission

30-01-2026

Department Of CS & IT

Faculty Of Computing and Engineering

University Of Kotli Azad Jammu and Kashmir

Contents

Abstract 4

1. Introduction 4

    1.1 Deadlock 4

2. Objectives 4

3. Deadlock Detection Approach 5

4. Working of Deadlock Detection Approach 5

5. Deadlock Conditions 6

    5.1.1 Mutual Exclusion: 6

    5.1.2 Hold and Wait 6

    5.1.3 No Preemption: 7

    5.1.4 Circular Wait: 7

6. Tools and Techniques Used 7

    6.1.1 Tools Used: 7

    6.1.2 Techniques Used: 7

7. Functional Requirements 8

8. Non-Functional Requirements 8

9. References 8

10. Conclusion 9

Abstract

Deadlock is a common problem in Operating Systems where two or more processes wait for each other to release resources, and as a result, none of them can continue. This project focuses on understanding and detecting deadlock using the Resource Allocation Graph approach. An online deadlock simulation tool is used to visually demonstrate how deadlock occurs between processes and resources. The main objective of this project is to explain the concept of deadlock detection in a simple and clear way so that students can easily understand how deadlock works in real systems.

1. Introduction

Deadlock occurs when a set of processes are blocked because each process holds a resource and waits for another. Understanding deadlock detection is essential for operating system design.

1.1 Deadlock is a situation in an operating system where two or more processes are waiting for resources held by each other.

Example

Process P1 holds Resource R1 and waits for Resource R2, while Process P2 holds Resource R2 and waits for Resource R1.

Since both processes wait forever, the system enters a deadlock state.

2. Objectives

- To understand the concept of deadlock
- To detect deadlock using Resource Allocation Graph
- To provide a visual and easy explanation of deadlock

3. Deadlock Detection Approach

Banker's Algorithm is mainly used for deadlock avoidance and requires prior knowledge of maximum resource requirements.

Since this project focuses on deadlock detection and visualization, the Resource Allocation Graph approach is used instead for simplicity and better understanding.

4. Working of Deadlock Detection Approach

This section explains how the Resource Allocation Graph approach is used to detect deadlock in the system.

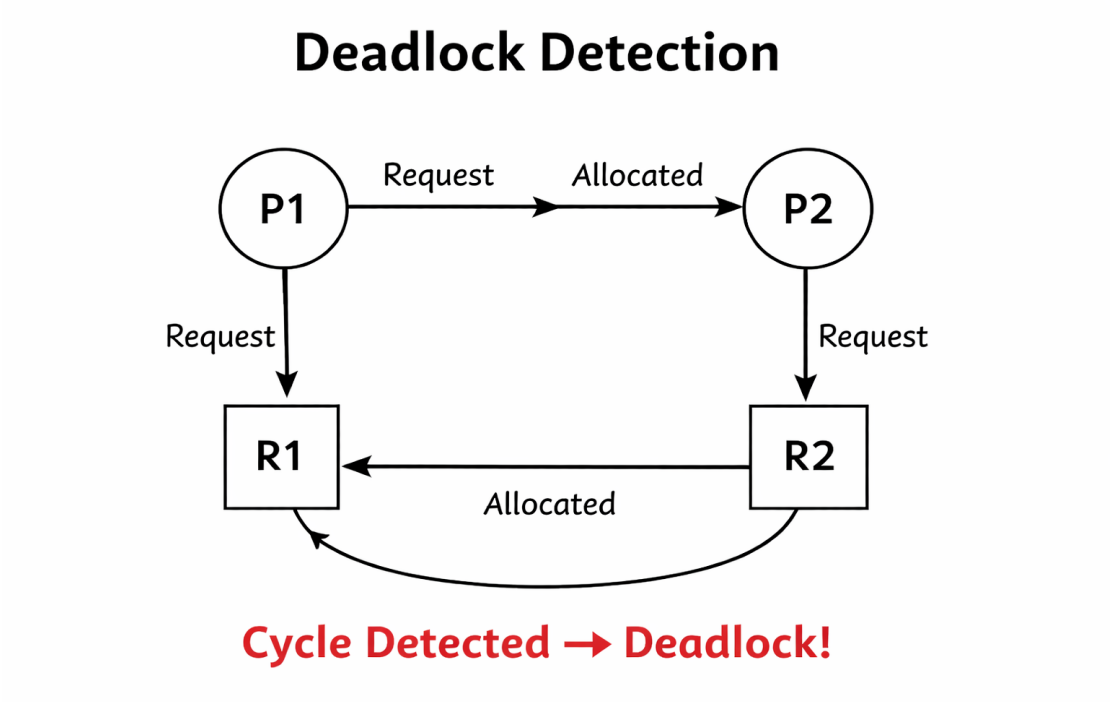
In this project, deadlock detection is performed using the Resource Allocation Graph approach.

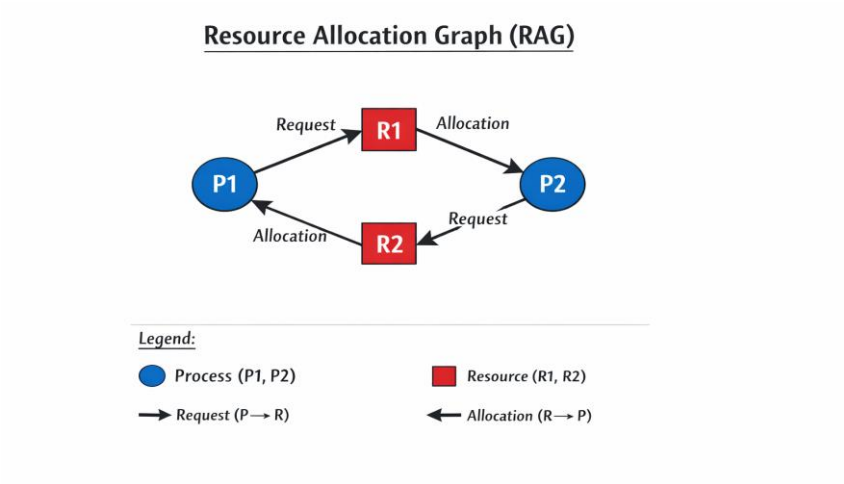
First, processes and resources are defined.

Then, request and allocation relationships are represented using directed edges.

Finally, the graph is analyzed to check for cycles.

If a cycle exists, the system detects a deadlock condition.





The above diagram represents a Resource Allocation Graph used for deadlock detection.

In the diagram, P1 and P2 represent processes, while R1 and R2 represent system resources.

Process P1 requests resource R1, which is currently allocated to process P2.

Similarly, process P2 requests resource R2, which is currently allocated to process P1.

This creates a circular wait condition in the graph.

Since a cycle exists in the Resource Allocation Graph, the system detects a deadlock condition**.**

5. Deadlock Conditions

Deadlock occurs in an operating system when the following four conditions occur at the same time.

5.1.1 Mutual Exclusion:

In mutual exclusion, a resource can be used by only one process at a time. If another process needs the same resource, it must wait until the resource is released.

5.1.2 Hold and Wait:

In this condition, a process holds at least one resource and at the same time waits for another resource that is currently held by another process.

5.1.3 No Preemption:

No preemption means that a resource cannot be taken forcibly from a process. The resource is released only when the process finishes its execution.

5.1.4 Circular Wait:

Circular wait occurs when a set of processes are waiting for resources in a circular chain. Each process waits for a resource that is held by the next process in the cycle.

Note: Deadlock occurs only when all four conditions are satisfied simultaneously.

6. Tools and Techniques Used

6.1.1 Tools Used:

Online Deadlock Simulator, Web Browser

Online Deadlock Simulation Tool was used to create and visualize the Resource Allocation Graph. A web browser (Google Chrome) was used to access the tool and demonstrate deadlock conditions. Basic Operating Sstem concepts were also used

for understanding the behaviour of processes and resources.

6.1.2 Techniques Used:

Resource Allocation Graph (RAG) for deadlock detection

The Resource Allocation Graph (RAG) technique was used to detect deadlock in the system. In this technique, processes and resources are represented as nodes, and directed edges are used to show resource requests and allocations.

Deadlock is detected by checking the presence of a cycle in the graph.

7. Functional Requirements

- The system shall allow the user to define processes and resources.
- The system shall create a Resource Allocation Graph.
- The system shall identify requests and allocations between processes and resources.
- The system shall detect deadlock by checking for cycles in the graph.
- The system shall display whether a deadlock exists or not.
- The user shall be able to modify resource allocation and recheck deadlock.

8. Non-Functional Requirements

- The system shall be easy to use and understand.
- The system shall provide results in a short time.
- The user interface shall be simple and student friendly.
- The system shall provide accurate deadlock detection results.
- The system shall be accessible through a web browser.
- The system documentation shall be clear and well-organized.

9. References

This is an online deadlock simulation tool used to visualize deadlock using a Resource Allocation Graph. It helps students understand how processes and resources can create a deadlock.

https://deadlock-mu.vercel.app/

10. Conclusion

This project successfully explains the concept of deadlock detection in an operating system using the Resource Allocation Graph approach. By representing processes and resources in a visual graph form, deadlock situations become easy to understand. The system detects deadlock by identifying cycles in the graph. This approach is simple, effective, and suitable for educational purposes. Overall, the project helps students clearly understand how deadlock occurs and how it can be detected in an operating system.